Consignes

Vous recevez une application qui gère un système de location de voitures. Trois modifications vous sont demandées.

Chaque modification proposée ci-dessous doit être implémentée et testée sur base de l'application fournie. A la fin de l'examen, vous rendrez donc 3 projets ainsi qu'une **brève description des modifications apportées**.

Ces 3 projets devront être déposés dans le dossier annoncé par votre professeur et être contenus dans un répertoire nommé en fonction de votre nom et de votre prénom.

Pour cette partie de l'examen, vous n'avez droit qu'à l'IDE NetBeans ainsi qu'à la *Javadoc* et à un éventuel aide-mémoire personnel.

Présentation de l'application

Votre application `LocationVoitures` permet à des loueurs de gérer et de facturer des locations de voitures à des clients.

Les voitures sont décrites par un numéro de chassis, un modèle et un prix journalier. Les clients possèdent un nom. Une location concerne un client et une voiture. Elle retient le nombre de kilomètres effectués et le nombre de jours loués.

Un loueur maintient la liste des locations en cours. Il veille à ce que deux voitures ne puissent pas être louées en même temps. Lorsqu'une location est terminée, on connait alors le nombre de jours et de kilomètres et le client se voit facturer le prix de la location. Par défaut, le prix d'une location est le nombre de jours loués multiplié par le prix journalier de la voiture.

Pour simplifier, on considérera qu'il n'y a pas deux objets identiques. Il est donc inutile de redéfinir `equals` et `hashCode`.

```
Loueur loueur = new Loueur();

Voiture citroenC1 = new Voiture("123", "Citroen C1", 75);
Voiture mercedesClasseE = new Voiture("789", "Mercredes Classe E", 150);

Client alain = new Client("Alain");
Client bruno = new Client("Bruno");

loueur.demarrerLocation(new Location(alain, citroenC1));
loueur.demarrerLocation(new Location(bruno, mercedesClasseE));

loueur.afficherLocations();

loueur.terminerLocation(citroenC1, 1800, 3);
loueur.terminerLocation(mercedesClasseE, 300, 4);

System.out.println(alain + " doit " + alain.getDette() + " €.");
System.out.println(bruno + " doit " + bruno.getDette() + " €.");
```

Par exemple, l'exécution du code ci-dessus donnerait l'affichage suivant :

```
Locations en cours :
 - Citroen C1 (123) louée à Alain
 - Mercredes Classe E (789) louée à Bruno
Alain doit 225.0 €.
Bruno doit 600.0 €.
```

Exécutez le programme fourni dans le dossier `Base` pour vous familiariser. Analysez attentivement le code et l'output produit.

Question 1 - Différents types de locations

Dans une évolution de notre application, on désire gérer plusieurs types de locations :

1. Les locations normales dont le prix est calculé par jour, quel que soit le nombre de kilomètres. Par exemple, pour louer une Citroen C1 à 75€par jour pendant 3 jours et 100km, le prix sera de 225€$(75 * 3)$.
2. Les locations "Alpha" dont le prix par jour est divisé par deux mais auquel on ajoute un prix par kilomètre correspondant à un deux-centième du prix par jour. Pour la même location, le prix sera donc de 150€$((75/2) * 3 + 100 * (75/200))$.
3. Les locations "Beta" dont le prix par jour est offert mais dont le prix au kilomètre correspond à un quarantième du prix par jour. Pour la même location, le prix sera donc de 187,5€$(100 * (75/40))$.

En partant de l'application `Locations`, et **sans modifier la méthode main()**, reproduisez **exactement** le même affichage que présenté ci dessous.

```
Loueur loueur = new Loueur();
Voiture citroenC1 = new Voiture("123", "Citroen C1", 75);
Client alain = new Client("Alain");

Location l1 = new Location(alain, citroenC1);
loueur.demarrerLocation(l1); loueur.terminerLocation(citroenC1, 100, 3);
System.out.println("Prix de l1 : " + l1.getPrix() + " €.");
System.out.println(alain + " doit " + alain.getDette() + " €.");

LocationAlpha l2 = new LocationAlpha(alain, citroenC1);
loueur.demarrerLocation(l2); loueur.terminerLocation(citroenC1, 100, 3);
System.out.println("Prix de l2 : " + l2.getPrix() + " €.");
System.out.println(alain + " doit " + alain.getDette() + " €.");

LocationBeta l3 = new LocationBeta(alain, citroenC1);
loueur.demarrerLocation(l3); loueur.terminerLocation(citroenC1, 100, 3);
System.out.println("Prix de l3 : " + l3.getPrix() + " €.");
System.out.println(alain + " doit " + alain.getDette() + " €.");
```

Le programme ci-dessus produit la sortie suivante :

```
Prix de l1 : 225.0 €.
Alain doit 225.0 €.
Prix de l2 : 150.0 €.
Alain doit 375.0 €.
Prix de l3 : 187.5 €.
Alain doit 562.5 €.
```

Bref résumé des modifications apportées :

Question 2 - Structure de données plus appropriée

La classe `Loueur` maintient une collection des locations en cours et fait en sorte qu'une voiture ne puisse pas être louée deux fois en même temps. Pour cela, elle utilise une `List<Location>`. Cette structure de donnée implique que le code des méthodes qui manipulent cette collection est compliqué. Remplacez cette `List<Location>` par une structure de données plus appropriée et modifiez les méthodes concernées.

Pour ce faire, modifiez l'application fournie dans le dossier `Structure` et complétez la pour reproduire **exactement** le comportement ci-dessous (seule l'ordre des locations affichées n'a pas d'importance). **Vous ne pouvez modifier que la classe Loueur !**.

```
Loueur loueur = new Loueur();

Voiture citroenC1 = new Voiture("123", "Citroen C1", 75);
Voiture mercedesClasseE = new Voiture("789", "Mercredes Classe E", 150);

Client alain = new Client("Alain");
Client bruno = new Client("Bruno");

loueur.demarrerLocation(new Location(alain, citroenC1));
loueur.demarrerLocation(new Location(bruno, mercedesClasseE));

try { loueur.demarrerLocation(new Location(bruno, mercedesClasseE)); }
catch (RuntimeException e) { System.out.println("Erreur : " + e.getMessage()); }

loueur.afficherLocations();
loueur.terminerLocation(citroenC1, 1800, 3);
loueur.demarrerLocation(new Location(alain, citroenC1));
loueur.terminerLocation(mercedesClasseE, 300, 4);

try { loueur.terminerLocation(mercedesClasseE, 100, 1); }
catch (RuntimeException e) { System.out.println("Erreur : " + e.getMessage()); }

loueur.afficherLocations();
```

L'exécution de ce morceau de code produit la sortie suivante :

```
Erreur : Voiture déjà louée
Locations en cours :
 - Citroen C1 (123) louée à Alain
 - Mercredes Classe E (789) louée à Bruno
Erreur : Location inexistante
Locations en cours :
 - Citroen C1 (123) louée à Alain
```

Bref résumé des modifications apportées :

Question 3 - Numéro de chassis unique

Dans l'application de base, il est possible de créer plusieurs véhicules avec le même numéro de chassis.

Arrangez-vous pour que la création d'une voiture avec un numéro de chassis déjà attribué soit impossible et déclenche une `ExceptionNumeroDeChassisExistant`, une exception *unchecked*.

En partant de l'application `ChassisUnique`, et **sans modifier la méthode main()**, reproduisez **exactement** le même affichage que présenté ci dessous.

```
Voiture citroenC1 = new Voiture("123", "Citroen C1", 75);
System.out.println("OK");
Voiture mercedesClasseE = new Voiture("789", "Mercredes Classe E", 150);
System.out.println("OK");
Voiture simca1000 = new Voiture("789", "Simca 1000", 5); //exception !
System.out.println("OK");
```

L'exécution de ce morceau de code produit la sortie suivante :

```
OK
OK
Exception in thread "main" chassisunique.ExceptionNumeroDeChassisExistant: 789
...
```

Bref résumé des modifications apportées :